```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
```

```
df=pd.read_csv('/content/supply_chain_data.csv')
df.head()
```

	Product type	SKU	Price	Availability	Number of products sold	Revenue generated	Customer demographics	Stock levels	Lead times	Order quantities	...	Location	Lead time	Pr
0	haircare	SKU0	69.808006	55	802	8661.996792	Non-binary	58	7	96	...	Mumbai	29	
1	skincare	SKU1	14.843523	95	736	7460.900065	Female	53	30	37	...	Mumbai	23	
2	haircare	SKU2	11.319683	34	8	9577.749626	Unknown	1	10	88	...	Mumbai	12	
3	skincare	SKU3	61.163343	68	83	7766.836426	Non-binary	23	13	59	...	Kolkata	24	
4	skincare	SKU4	4.805496	26	871	2686.505152	Non-binary	5	3	56	...	Delhi	5	

5 rows × 24 columns

```
df.shape,df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 24 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Product type             100 non-null    object
 1   SKU                      100 non-null    object
 2   Price                    100 non-null    float64
 3   Availability             100 non-null    int64
 4   Number of products sold  100 non-null    int64
 5   Revenue generated        100 non-null    float64
 6   Customer demographics    100 non-null    object
 7   Stock levels             100 non-null    int64
 8   Lead times               100 non-null    int64
 9   Order quantities         100 non-null    int64
 10  Shipping times           100 non-null    int64
 11  Shipping carriers        100 non-null    object
 12  Shipping costs           100 non-null    float64
 13  Supplier name            100 non-null    object
 14  Location                 100 non-null    object
 15  Lead time                100 non-null    int64
 16  Production volumes       100 non-null    int64
 17  Manufacturing lead time  100 non-null    int64
 18  Manufacturing costs      100 non-null    float64
 19  Inspection results       100 non-null    object
 20  Defect rates             100 non-null    float64
 21  Transportation modes     100 non-null    object
 22  Routes                   100 non-null    object
 23  Costs                    100 non-null    float64
dtypes: float64(6), int64(9), object(9)
memory usage: 18.9+ KB
((100, 24), None)
```

```
df.describe()
```

	Price	Availability	Number of products sold	Revenue generated	Stock levels	Lead times	Order quantities	Shipping times	Shipping costs	Lead time	Pr
count	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000	10
mean	49.462461	48.400000	460.990000	5776.048187	47.770000	15.960000	49.220000	5.750000	5.548149	17.080000	56
std	31.168193	30.743317	303.780074	2732.841744	31.369372	8.785801	26.784429	2.724283	2.651376	8.846251	26
min	1.699976	1.000000	8.000000	1061.618523	0.000000	1.000000	1.000000	1.000000	1.013487	1.000000	10
25%	19.597823	22.750000	184.250000	2812.847151	16.750000	8.000000	26.000000	3.750000	3.540248	10.000000	35
50%	51.239831	43.500000	392.500000	6006.352023	47.500000	17.000000	52.000000	6.000000	5.320534	18.000000	56
75%	77.198228	75.000000	704.250000	8253.976921	73.000000	24.000000	71.250000	8.000000	7.601695	25.000000	79

Data cleaning

```
df.isnull().sum().sum()
# No null value is present
```

```
np.int64(0)
```

```
df.duplicated().sum() # No duplicated value is present
```

```
np.int64(0)
```

```
df.columns
```

```
Index(['Product type', 'SKU', 'Price', 'Availability',
       'Number of products sold', 'Revenue generated', 'Customer demographics',
       'Stock levels', 'Lead times', 'Order quantities', 'Shipping times',
       'Shipping carriers', 'Shipping costs', 'Supplier name', 'Location',
       'Lead time', 'Production volumes', 'Manufacturing lead time',
       'Manufacturing costs', 'Inspection results', 'Defect rates',
       'Transportation modes', 'Routes', 'Costs'],
      dtype='object')
```

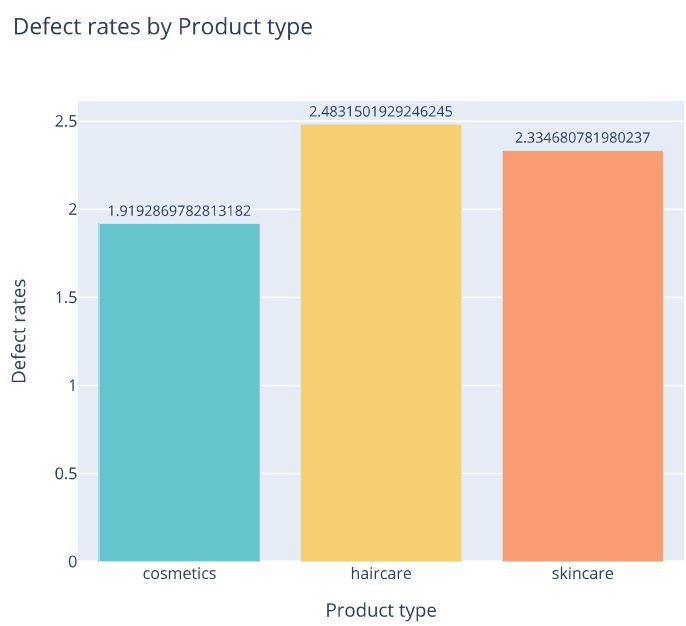
EDA

```
# Select only categorical (object/string) columns
categorical_cols = df.select_dtypes(include=['object', 'category']).columns

# Print unique values for each categorical column
for col in categorical_cols:
    print(f"Column: {col}")
    print(df[col].unique())
```

```
Column: Product type
['haircare' 'skincare' 'cosmetics']
Column: SKU
['SKU0' 'SKU1' 'SKU2' 'SKU3' 'SKU4' 'SKU5' 'SKU6' 'SKU7' 'SKU8' 'SKU9'
 'SKU10' 'SKU11' 'SKU12' 'SKU13' 'SKU14' 'SKU15' 'SKU16' 'SKU17' 'SKU18'
 'SKU19' 'SKU20' 'SKU21' 'SKU22' 'SKU23' 'SKU24' 'SKU25' 'SKU26' 'SKU27'
 'SKU28' 'SKU29' 'SKU30' 'SKU31' 'SKU32' 'SKU33' 'SKU34' 'SKU35' 'SKU36'
 'SKU37' 'SKU38' 'SKU39' 'SKU40' 'SKU41' 'SKU42' 'SKU43' 'SKU44' 'SKU45'
 'SKU46' 'SKU47' 'SKU48' 'SKU49' 'SKU50' 'SKU51' 'SKU52' 'SKU53' 'SKU54'
 'SKU55' 'SKU56' 'SKU57' 'SKU58' 'SKU59' 'SKU60' 'SKU61' 'SKU62' 'SKU63'
 'SKU64' 'SKU65' 'SKU66' 'SKU67' 'SKU68' 'SKU69' 'SKU70' 'SKU71' 'SKU72'
 'SKU73' 'SKU74' 'SKU75' 'SKU76' 'SKU77' 'SKU78' 'SKU79' 'SKU80' 'SKU81'
 'SKU82' 'SKU83' 'SKU84' 'SKU85' 'SKU86' 'SKU87' 'SKU88' 'SKU89' 'SKU90'
 'SKU91' 'SKU92' 'SKU93' 'SKU94' 'SKU95' 'SKU96' 'SKU97' 'SKU98' 'SKU99']
Column: Customer demographics
['Non-binary' 'Female' 'Unknown' 'Male']
Column: Shipping carriers
['Carrier B' 'Carrier A' 'Carrier C']
Column: Supplier name
['Supplier 3' 'Supplier 1' 'Supplier 5' 'Supplier 4' 'Supplier 2']
Column: Location
['Mumbai' 'Kolkata' 'Delhi' 'Bangalore' 'Chennai']
Column: Inspection results
['Pending' 'Fail' 'Pass']
```

```
Column: Transportation modes
['Road' 'Air' 'Rail' 'Sea']
Column: Routes
['Route B' 'Route C' 'Route A']
```

```
import plotly.express as px

# gives the mean of the defetct rates of each product
defect_rates_by_product = df.groupby('Product type')['Defect rates'].mean().reset_index()

fig = px.bar(
    defect_rates_by_product,
    x='Product type',
    y='Defect rates',

    title='Defect rates by Product type',
    text='Defect rates'     # 👈 ADD THIS LINE
)

color_scale=px.colors.qualitative.Pastel
fig.update_traces(marker_color=color_scale)
fig.update_traces(textposition='outside')  # 👈 to show text above bars
fig.update_layout(uniformtext_minsize=8, uniformtext_mode='hide')

fig.show()
```



- calculate and plot the risk score

```
# supply chain risk assessment

risk_data=df[['SKU','Lead times','Stock levels']]
print(risk_data.head())

# risk score tells us how likely we are to run out of stock
risk_data['risk score'] = risk_data['Lead times'] + (1-risk_data.loc[:,'Stock levels'])
risk_data.head()
```

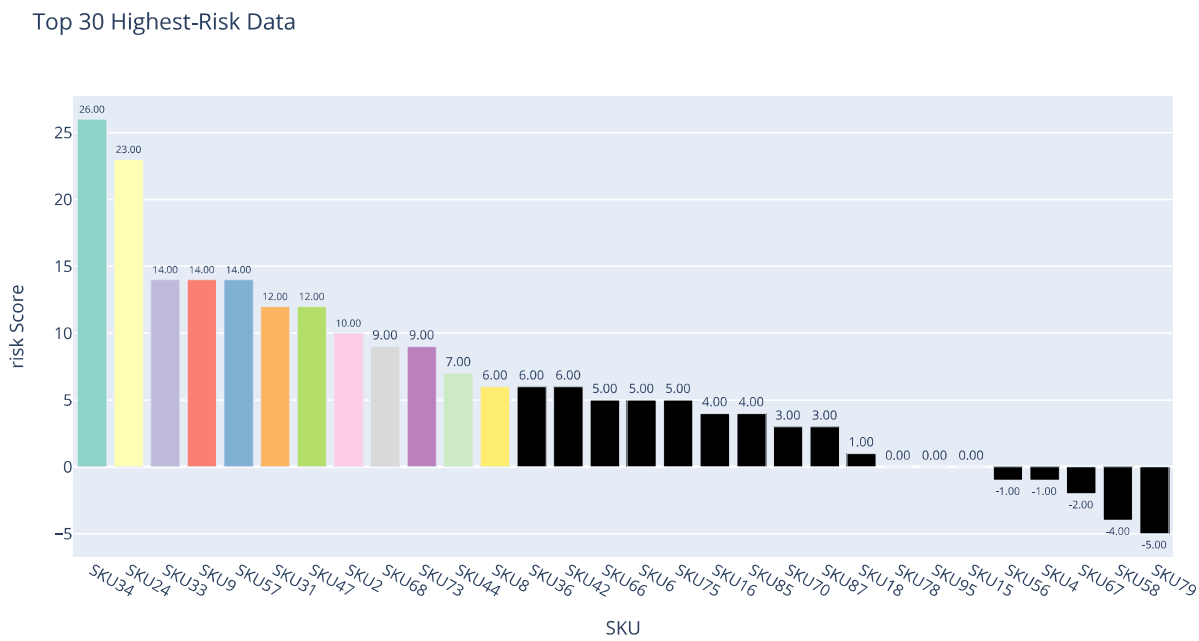
```
    SKU  Lead times  Stock levels
0  SKU0           7            58
1  SKU1          30            53
2  SKU2          10             1
3  SKU3          13            23
4  SKU4           3             5
/tmp/ipython-input-1366182897.py:7: SettingWithCopyWarning:


A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-ver
```

	SKU	Lead times	Stock levels	risk score
0	SKU0	7	58	-50
1	SKU1	30	53	-22
2	SKU2	10	1	10
3	SKU3	13	23	-9
4	SKU4	3	5	-1

```
risk_data=risk_data.sort_values(by='risk score',ascending=False)[:30]
risk_data.head()
```

	SKU	Lead times	Stock levels	risk score
34	SKU34	26	1	26
24	SKU24	26	4	23
33	SKU33	17	4	14
9	SKU9	27	14	14
57	SKU57	18	5	14

```
# Create a bar plot using Plotly Express
fig = px.bar(risk_data, x='SKU', y='risk score', title='Top 30 Highest-Risk Data',
             labels={'risk score': 'risk Score', 'SKU': 'SKU'},
             text='risk score')

# Customize the appearance of the plot
color_scale=px.colors.qualitative.Set3
fig.update_traces(marker_color=color_scale)
fig.update_traces(texttemplate='%{text:.2f}', textposition='outside')

# Show the plot
fig.show()

# positive value suggest the stockout risk , zero value: " Stock is jusr be enough " , negative: "have extra stock"
```



- EOQ (Economic Order Quality)
- EOQ tells how much quantity we should order at one time so that our ordering cost is low , holding cost is low and we nevwe run out of stock

```
holdingcost=0.2
```

```
def calculate_eoq(data):

  S=data['Costs']
  D=data['Number of products sold']
  H=data['Number of products sold'] * holdingcost
  EOQ=np.sqrt((2*D*S)/H)
  return round(EOQ,2)
```

```
df['EOQ']=calculate_eoq(df)
df.head()
```

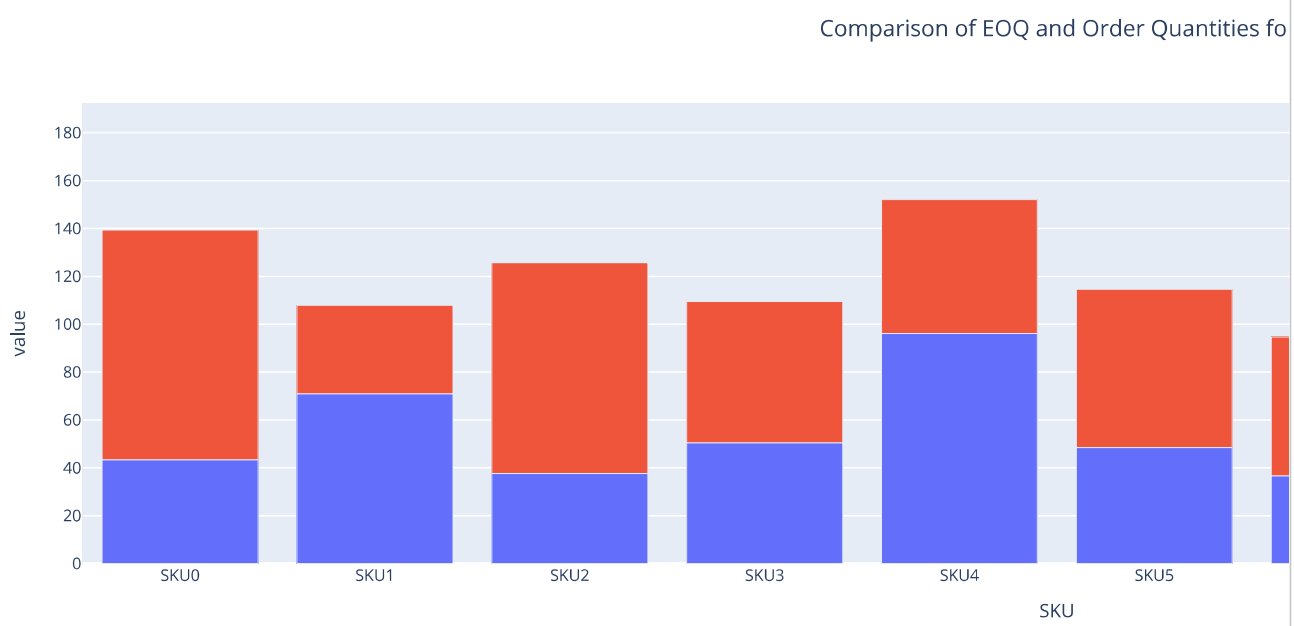
	Product type	SKU	Price	Availability	Number of products sold	Revenue generated	Customer demographics	Stock levels	Lead times	Order quantities	...	Lead time	Production volumes
0	haircare	SKU0	69.808006	55	802	8661.996792	Non-binary	58	7	96	...	29	215
1	skincare	SKU1	14.843523	95	736	7460.900065	Female	53	30	37	...	23	517
2	haircare	SKU2	11.319683	34	8	9577.749626	Unknown	1	10	88	...	12	971
3	skincare	SKU3	61.163343	68	83	7766.836426	Non-binary	23	13	59	...	24	937
4	skincare	SKU4	4.805496	26	871	2686.505152	Non-binary	5	3	56	...	5	414

5 rows × 25 columns

```
comparison_columns = df[['SKU', 'EOQ','Order quantities']]
comparison_columns.head()
```

	SKU	EOQ	Order quantities
0	SKU0	43.33	96
1	SKU1	70.93	37
2	SKU2	37.67	88
3	SKU3	50.48	59
4	SKU4	96.10	56

```
# Extracting the top 10 rows for comparison
top_10_comparison = comparison_columns.head(10)

# Creating a bar plot to compare EOQ and Order Quantities for the top 10 SKUs
fig = px.bar(top_10_comparison, x='SKU', y=['EOQ', 'Order quantities'],
             title='Comparison of EOQ and Order Quantities for Top 10 SKUs')

fig.update_layout(
    title_x=0.5
)

# Displaying the plot
fig.show()
```

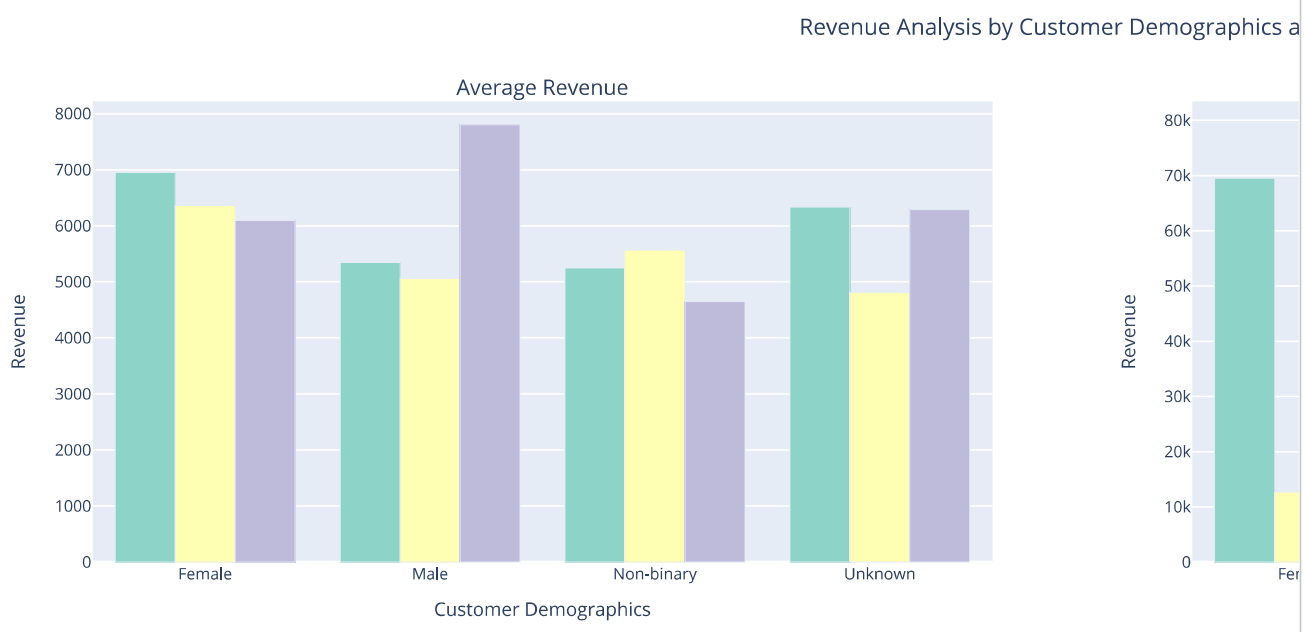


- CUSTOMER SEGMENTATION ANALYSIS

```
revenue_avg_by_demo_prod = df.groupby(['Customer demographics', 'Product type'])['Revenue generated'].mean().reset_index()
revenue_sum_by_demo_prod = df.groupby(['Customer demographics', 'Product type'])['Revenue generated'].sum().reset_index()
```

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go

# Create a palette of different colors for each product type
colors = px.colors.qualitative.Set3

# Create a subplot with 1 row and 2 columns
fig = make_subplots(rows=1, cols=2, subplot_titles=('Average Revenue', 'Total Revenue'))

# Plot for average revenue
for i, product_type in enumerate(revenue_avg_by_demo_prod['Product type'].unique()):
    subset = revenue_avg_by_demo_prod[revenue_avg_by_demo_prod['Product type'] == product_type]
    fig.add_trace(
        go.Bar(
            x=subset['Customer demographics'],
```

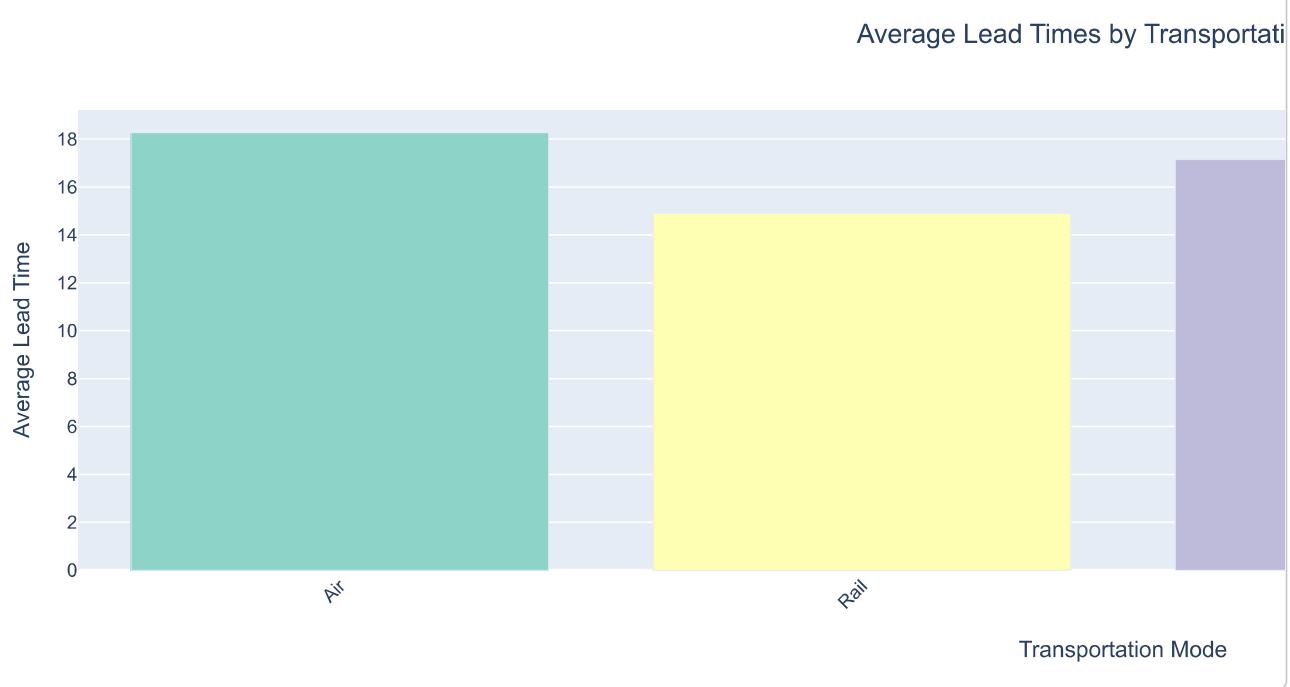
```
            y=subset['Revenue generated'],
            name=product_type,
            marker_color=colors[i]
        ),
        row=1, col=1
    )

# Plot for total revenue without legend
for i, product_type in enumerate(revenue_sum_by_demo_prod['Product type'].unique()):
    subset = revenue_sum_by_demo_prod[revenue_sum_by_demo_prod['Product type'] == product_type]
    fig.add_trace(
        go.Bar(
            x=subset['Customer demographics'],
            y=subset['Revenue generated'],
            showlegend=False,  # Hide legend for this subplot
            marker_color=colors[i]
        ),
        row=1, col=2
    )

# Beautify the subplot and show the legend for the first subplot
fig.update_layout(
    title='Revenue Analysis by Customer Demographics and Product Type',
    xaxis=dict(title='Customer Demographics'),
    yaxis=dict(title='Revenue'),
    xaxis2=dict(title='Customer Demographics'),
    yaxis2=dict(title='Revenue'),
    title_x=0.5,
    showlegend=True  # Show legend for the first subplot
)

# Show the subplot
fig.show()
```



- LEAD TIMES OPTIMIZATION ANALYSIS

```
avg_lead_times_transport = df.groupby(['Transportation modes'])['Lead times'].mean().reset_index()

fig = px.bar(avg_lead_times_transport, x='Transportation modes', y='Lead times',
             labels={'Transportation modes': 'Transportation Mode', 'Lead times': 'Average Lead Time'},
             title='Average Lead Times by Transportation Mode')

# Customize the color scale for bars
color_scale = px.colors.qualitative.Set3  # You can choose a different color scale
fig.update_traces(marker_color=color_scale)

# Customize the layout for beautification
```

```
fig.update_layout(
    title=dict(text='Average Lead Times by Transportation Mode', x=0.5),
    xaxis_title='Transportation Mode',
    yaxis_title='Average Lead Time',
    xaxis_tickangle=-45,  # Rotate x-axis labels for better readability
    font=dict(family="Arial", size=14),
    showlegend=False  # Remove legend
)

# Show the plot
fig.show()
```




```
best_transportation_mode = avg_lead_times_transport.loc[avg_lead_times_transport['Lead times'].idxmin()]
best_transportation_mode
```

	3
Transportation modes	Sea
Lead times	12.176471

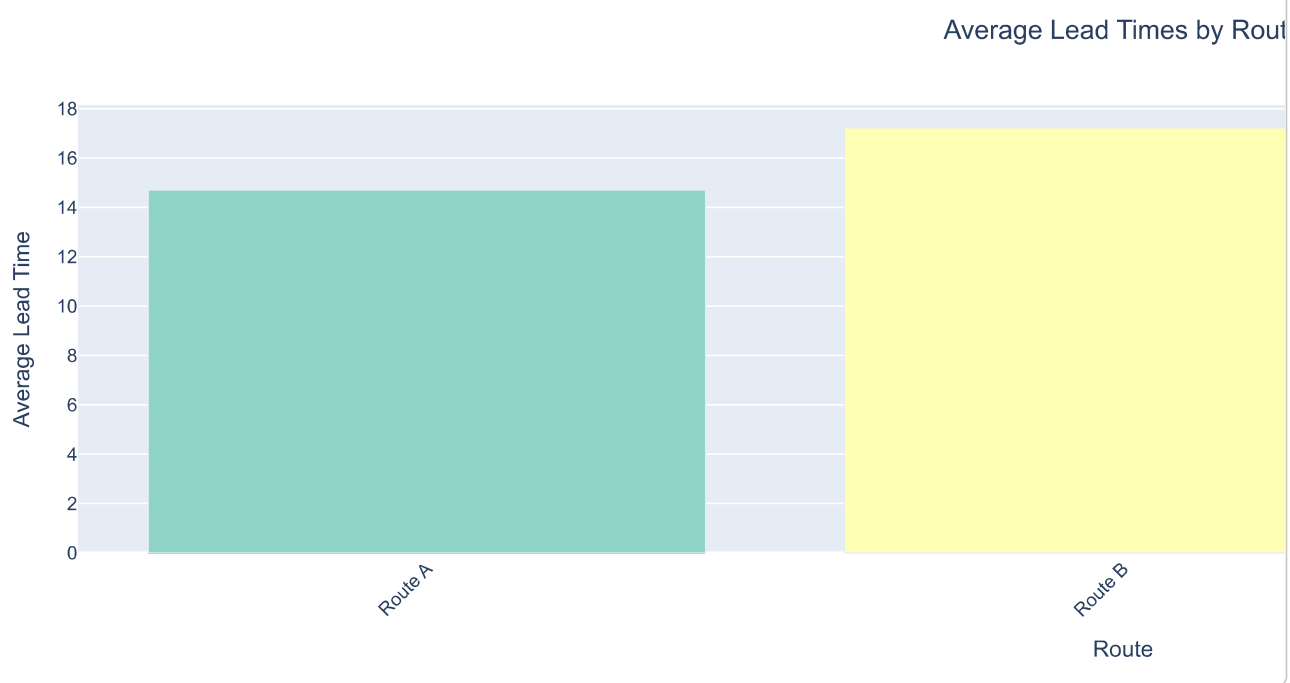
dtype: object

```
best_transportation_data = df[df['Transportation modes']==best_transportation_mode['Transportation modes']]
best_transportation_data.head()
```

	Product type	SKU	Price	Availability	Number of products sold	Revenue generated	Customer demographics	Stock levels	Lead times	Order quantities	...	Lead time	Producti volum
6	skincare	SKU6	4.078333	48	65	7823.476560	Male	11	15	58	...	14	3
8	cosmetics	SKU8	68.717597	78	150	7517.363211	Female	5	10	15	...	13	7
15	skincare	SKU15	36.989245	94	469	5442.086785	Non-binary	9	8	69	...	14	5
18	haircare	SKU18	36.443628	23	620	9364.673505	Unknown	10	10	46	...	18	3
27	cosmetics	SKU27	92.557361	42	352	2686.457224	Unknown	47	9	62	...	25	2

5 rows × 25 columns

```
import plotly.express as px

avg_lead_times_route = df.groupby(['Routes'])['Lead times'].mean().reset_index()

# Assuming you have the avg_lead_times_route DataFrame already defined
fig = px.bar(avg_lead_times_route, x='Routes', y='Lead times',
             labels={'Routes': 'Route', 'Lead times': 'Average Lead Time'},
             title='Average Lead Times by Route')

# Customize the color scale for bars
color_scale = px.colors.qualitative.Set3  # You can choose a different color scale
fig.update_traces(marker_color=color_scale)

# Customize the layout for beautification
fig.update_layout(
    title=dict(text='Average Lead Times by Route', x=0.5),
    xaxis_title='Route',
    yaxis_title='Average Lead Time',
    xaxis_tickangle=-45,  # Rotate x-axis labels for better readability
    font=dict(family="Arial", size=14),
    showlegend=False  # Remove legend
)

# Show the plot
fig.show()
```




```
best_route = avg_lead_times_route.loc[avg_lead_times_route['Lead times'].idxmin()]
best_route
```

	0
Routes	Route A
Lead times	14.697674

dtype: object

```
best_route_data = df[df['Routes']==best_route['Routes']]
best_route_data.head()
```

	Product type	SKU	Price	Availability	Number of products sold	Revenue generated	Customer demographics	Stock levels	Lead times	Order quantities	...	Lead time	Productio volume
3	skincare	SKU3	61.163343	68	83	7766.836426	Non-binary	23	13	59	...	24	937
4	skincare	SKU4	4.805496	26	871	2686.505152	Non-binary	5	3	56	...	5	414
5	haircare	SKU5	1.699976	87	147	2828.348746	Non-binary	90	27	66	...	10	104
6	skincare	SKU6	4.078333	48	65	7823.476560	Male	11	15	58	...	14	314
11	skincare	SKU11	90.635460	95	960	6099.944116	Female	46	23	60	...	28	362

5 rows × 25 columns

- MODELLING

```
X = df.loc[:,['Price', 'Availability', 'Stock levels', 'Lead times', 'Order quantities']]
y = df.loc[:,'Number of products sold']
```

```
#num of folds for cross_validation
num_folds=10

# Initialize lists to store evaluation metrics
mse_scores = []
rmse_scores = []
mae_scores = []
r2_scores = []
```

```
from sklearn.model_selection import KFold

# Create a KFold object
kf = KFold(n_splits=num_folds, shuffle=True, random_state=42)
```

```
# Define the parameters for LightGBM
params = {
    'objective': 'regression',
    'metric': 'mean_squared_error',
    'boosting_type': 'gbdt',
    'num_leaves': 31,
    'learning_rate': 0.05,
    'feature_fraction': 0.9
}
```

```
import lightgbm as lgb
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

for train_index, test_index in kf.split(X):
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

    # Create LightGBM datasets for training and testing
    train_data = lgb.Dataset(X_train, label=y_train)
    test_data = lgb.Dataset(X_test, label=y_test, reference=train_data)

    # Add early stopping
    num_round = 100  # Increase the number of boosting rounds
    early_stopping_rounds = 5  # Set the number of rounds to wait for early stopping

    # Train the model with early stopping
    bst = lgb.train(
        params,
        train_data,
        num_round,
        valid_sets=[test_data],
        callbacks=[lgb.early_stopping(stopping_rounds=early_stopping_rounds, verbose=False)]
```

```
    )

    # Make predictions on the test set
    y_pred = bst.predict(X_test, num_iteration=bst.best_iteration)

    # Calculate metrics
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    # Calculate the target range
    target_range = np.max(y_test) - np.min(y_test)

    # Calculate metrics as percentages
    percentage_mse = (mse / target_range) * 100
    percentage_rmse = (rmse / target_range) * 100
    percentage_mae = (mae / target_range) * 100
    percentage_r2 = (r2 * 100)

    # Append the scores to the respective lists
    mse_scores.append(percentage_mse)
    rmse_scores.append(percentage_rmse)
    mae_scores.append(percentage_mae)
    r2_scores.append(percentage_r2)
```

Show hidden output

```
# Calculate the average scores over all folds
avg_mse = np.mean(mse_scores)
avg_rmse = np.mean(rmse_scores)
avg_mae = np.mean(mae_scores)
avg_r2 = np.mean(r2_scores)

# Print the results
print(f"Average Mean Squared Error: {avg_mse:.2f}%")
print(f"Average Root Mean Squared Error: {avg_rmse:.2f}%")
print(f"Average Mean Absolute Error: {avg_mae:.2f}%")
print(f"Average R-squared: {avg_r2:.2f}%")
```

```
Average Mean Squared Error: 10450.61%
Average Root Mean Squared Error: 34.70%
Average Mean Absolute Error: 31.02%
Average R-squared: -4.64%
```

- COST OPTIMIZATION

```
# Extract the features (X) and target (y)
X = df.loc[:, 'Production volumes'].values.reshape(-1, 1)
y = df.loc[:, 'Manufacturing costs'].values
```

```
# Define the number of folds for cross-validation
num_folds = 5
```

```
# Initialize lists to store evaluation metrics
mse_scores = []
rmse_scores = []
mae_scores = []
r2_scores = []
```

```
from sklearn.preprocessing import MinMaxScaler

# Initialize and fit the scaler to the data
scaler = MinMaxScaler()
scaler.fit(X)
```

```
▾ MinMaxScaler
MinMaxScaler()
```

```
# Create a KFold object
kf = KFold(n_splits=num_folds, shuffle=True, random_state=42)
```

```
import tensorflow as tf
from tensorflow.keras.layers import Dense
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

# Define and compile the neural network model outside the loop to prevent re-tracing
model = tf.keras.Sequential([
    Input(shape=(1,)), # Recommended way to define input shape
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(1)
])

model.compile(optimizer='adam', loss='mean_squared_error')

# Define Early Stopping callback
early_stopping = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

# Define Model Checkpoint callback to save the best model
model_checkpoint = ModelCheckpoint('/kaggle/working/best_model.weights.h5', save_best_only=True, save_weights_only=True, monitc

for train_index, test_index in kf.split(X):
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]

    # Scale the data
    X_train_scaled = scaler.transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # Define and compile the neural network model
    model = tf.keras.Sequential([
        Dense(64, activation='relu', input_dim=1),
        Dense(32, activation='relu'),
        Dense(1)
    ])

    model.compile(optimizer='adam', loss='mean_squared_error')

    # Define Early Stopping callback
    early_stopping = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

    # Define Model Checkpoint callback to save the best model
    model_checkpoint = ModelCheckpoint('/kaggle/working/best_model.weights.h5', save_best_only=True, save_weights_only=True, mc

    # Train the model on the training data with callbacks
    history = model.fit(X_train_scaled,
                        y_train, epochs=100,
                        batch_size=32,
                        validation_data=(X_test_scaled, y_test),
                        callbacks=[early_stopping, model_checkpoint])

    # Load the best model weights from the saved checkpoint
    model.load_weights('/kaggle/working/best_model.weights.h5') # Corrected filepath

    # Make predictions on the test set
    y_pred = model.predict(X_test_scaled)

    # Calculate metrics
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    mae = np.mean(np.abs(y_test - y_pred))
    r2 = r2_score(y_test, y_pred)

    # Calculate metrics as percentages
    target_range = np.max(y_test) - np.min(y_test)
    percentage_mse = (mse / target_range) * 100
    percentage_rmse = (rmse / target_range) * 100
    percentage_mae = (mae / target_range) * 100
    percentage_r2 = (r2 * 100)

    # Append the scores to the respective lists
    mse_scores.append(percentage_mse)
    rmse_scores.append(percentage_rmse)
    mae_scores.append(percentage_mae)
```

```
    r2_scores.append(percentage_r2)
```

Show hidden output

```
# Calculate the average scores over all folds
avg_mse = np.mean(mse_scores)
avg_rmse = np.mean(rmse_scores)
avg_mae = np.mean(mae_scores)
avg_r2 = np.mean(r2_scores)

# Print the results
print(f"Average Mean Squared Error: {avg_mse:.2f}%")
print(f"Average Root Mean Squared Error: {avg_rmse:.2f}%")
print(f"Average Mean Absolute Error: {avg_mae:.2f}%")
print(f"Average R-squared: {avg_r2:.2f}%")
```